A Distributed Intelligence Architecture for Responsive Command Handling in Edge Robotics

Wesley W. Lowman*
NASA Marshall Space Flight Center, Huntsville, AL 35808

Donovan H. Ho† and Aanchal L. Save‡
Amentum, Huntsville, AL 35806

Jonathan T. Breckenridge§ and Tanner J. Bonds¶
NASA Marshall Space Flight Center, Huntsville, AL 35808

Communication-constrained environments such as those that require Earth independence demand robotic autonomy that functions without reliance on cloud infrastructure. Current approaches to robotic control in these settings often depend on deterministic procedures that lack the flexibility to adapt to unstructured tasks or natural language commands. This paper presents REACH (Responsive Embedded Assistant for Command Handling), an in-house-developed distributed edge intelligence architecture integrating large language model (LLM) task planning and computer vision on modules connected through a shared communication framework. REACH enables natural language-driven robotic autonomy in edge computing environments without reliance on cloud-based data storage and computational resources. REACH employs an architecture of four primary modules in which a locally hosted LLM translates natural language commands into structured robotic procedures, enabling consistent and interpretable task execution. This is paired with perception functionality, supporting real-time object localization in resource-constrained environments. Experimental results show average end-to-end command processing latency of 0.642 seconds for text input and 2.396 seconds for voice input, with a 98.3% single-step command success rate and an 86.8% multi-step task success rate. The system maintains full functionality without any external network connectivity, demonstrating the practicality of LLM-driven robotic autonomy for resource-constrained mission environments.

Nomenclature

A_m	=	center position of fiducial marker m, m
H	=	homography matrix
K	=	camera intrinsic matrix
$\mathbf{O}$	=	world-coordinate origin
h_{ij}	=	element of homography matrix (row i, column j)
m	=	fiducial marker index
n	=	sample count
t	=	latency measurement, s
$\tilde{t}$	=	sample median latency, s
t^*_{del}	=	corrected delivery latency, s
t_{recv}	=	message receive timestamp, s
t_{send}	=	message send timestamp, s
(u, v)	=	pixel coordinates in image frame
w	=	homogeneous normalization factor

*Implementation & Test Engineer, Spacecraft and Vehicle Systems Department.
†Robotics Engineering Intern, Spacecraft and Vehicle Systems Department.
‡Robotics Engineering Intern, Spacecraft and Vehicle Systems Department.
§Implementation & Test Team Lead, Spacecraft and Vehicle Systems Department.
¶Implementation & Test Engineer, Spacecraft and Vehicle Systems Department.

(x, y, z)	=	world coordinates, m
$(\hat{x}_i, \hat{y}_i)$	=	estimated coordinates of sample i, m
(x_i, y_i)	=	ground-truth coordinates of sample i, m
$\mathcal{A}$	=	set of fiducial markers
$\mathcal{D}_{\text{valid}}$	=	set of valid latency measurements
δ	=	detected object
$\delta_{I,c}$	=	detected object c in image coordinates (homogeneous)
δ_W	=	detected object position in world coordinates
Δ_{clock}	=	NTP clock offset between compute modules, s
ϵ_i	=	localization error of sample i, cm

I. Introduction

Robotic autonomy has long been a cornerstone of space exploration, enabling missions to perform complex operations under communication delays or without direct human intervention. Traditionally, autonomy has been achieved through deterministic planning methods that implement rigidly structured software. While robust, such approaches require exhaustive pre-programming, lack adaptability to unexpected scenarios, and depend heavily on continuous oversight, often requiring ground-based supervision. These characteristics are misaligned with future space exploration goals—specifically in-space servicing, assembly, and manufacturing (ISAM), Earth-independent operations (EIO), and human-robot teaming—where flexibility and resilience are critical factors for mission success.

Recent advancements in large language models (LLMs), including open-source model availability and compute efficiency, have opened new avenues for robotic autonomy. Unlike deterministic frameworks, LLMs excel at parsing unstructured human input, enabling natural language-driven tasking. Meanwhile, advances in edge hardware platforms allow LLM inference at the point of use, eliminating dependence on cloud-based resources. Within this paper, "edge" refers to a location at or near the source of the data on a network as opposed to a remote location that a cloud solution provides. Combined, these technologies open the door to resilient, distributed autonomy solutions suitable for mission settings where external connectivity is constrained or unavailable.

This paper introduces REACH (Responsive Embedded Assistant for Command Handling), an in-house-developed modular architecture that integrates natural language tasking, edge-resident LLM inference, computer vision, and robotic control into a framework for distributed intelligence. By operating entirely at the edge, REACH provides robust, low-latency autonomy suitable for mission scenarios requiring independence from Earth-based resources or environments in which external variables frequently change.

This work builds upon advances in LLM-driven multimodal robotics and neural network-based kinematics, both reviewed in Section II. Together, these foundations enable the progression from motion-level learning to task-level planning within a distributed, edge-based architecture for mission settings.

The primary contributions of this work are:

1) A fully local, modular architecture for distributed intelligence and LLM-driven robotic autonomy at the edge,
2) A natural language translation framework that produces structured, executable robotic commands,
3) Integration of perception, language-driven tasking, and control within a distributed edge system, and
4) Experimental validation of latency, reliability, and robustness under internal faults.

The remainder of this paper is organized as follows. Sections II and III review background information and present the REACH system architecture. Section IV describes the REACH implementation, Section V reports experimental results, and Section VI provides discussion. Finally, Sections VII and VIII outline future work and conclude the paper.

II. Background and Related Work

The REACH architecture draws on several areas of research that motivate and inform its design. The following subsections review related work across five areas: LLM-driven robotics, edge computing for robotic systems, neural network-based kinematics, prompt engineering for robotic command generation, and perception systems for robotic manipulation. For each area, relevant prior work is summarized and the specific gap or opportunity that REACH addresses is identified.

A. LLM-Driven Robotics

Large language models have rapidly expanded from natural language processing into embodied robotic systems. Robotic Transformer 2 (RT-2) [1], developed by Google, jointly trained a vision-language-action model by tokenizing robotic trajectories and embedding them within the same output space as natural language, enabling a single model to perform web-scale vision-language understanding alongside low-level robot control. Across approximately 6,000 evaluation trials, RT-2 generalized to novel objects and instructions not present in its robotic training data, including multi-step reasoning chains that combined semantic understanding with physical manipulation. PaLM-E [2] ingested combined sequences of text, images, and continuous robot state representations, processing them through a single large language model trained end-to-end to output plans for sequential manipulation tasks. With configurations as large as 562 billion parameters, PaLM-E improved performance across all evaluated task categories after training jointly on web text, visual data, and robotic experience. This indicates that multi-domain pre-training benefits embodied reasoning. The ELLMER framework paired GPT-4 with retrieval-augmented generation (RAG) so that a manipulator could autonomously plan and adapt to environmental disturbances during multi-step tasks, including coffee preparation and plate decoration, using force and visual feedback [3].

While these systems represent significant advances, they rely heavily on cloud-based infrastructure for both training and inference. This dependence introduces latency, bandwidth requirements, and data sovereignty concerns that limit their applicability in communication-constrained environments such as space missions. REACH addresses this limitation by hosting LLM inference entirely on edge devices, eliminating reliance on external compute while preserving the ability to parse unstructured natural language commands.

B. Edge Computing in Robotics

Edge computing enables low-latency processing by placing computation near data sources, reducing reliance on remote servers and improving system resilience. These qualities make it well suited for space applications where round-trip communication delays and bandwidth constraints preclude cloud-based solutions. However, Janbi et al. [4] noted in a survey of the distributed AI landscape that AI workloads require significant compute while edge platforms remain resource-constrained, underscoring the need for architectures that balance local processing with the demands of modern inference. A modular architecture can navigate this constraint by distributing specialized workloads across independent edge nodes, each sized to its task.

Several recent efforts have applied edge computing specifically to robotic systems. Gala et al. [5] proposed a containerized edge architecture using Linux, Docker, and Kubernetes over a time-triggered wireless network to migrate a safety-critical robotic localization task off dedicated onboard hardware, with calculated upper bounds on network transmission time. In a complementary direction, Sikorski et al. [6] combined a cloud-hosted language model with edge-resident vision and speech modules on a robotic arm, reporting successful spoken-command-driven object manipulation in an assistive robotics context.

However, none of these systems host LLM inference directly on edge hardware. REACH closes this gap by running the language model, speech recognition, object detection, and robotic control entirely on local modules connected through a dedicated network switch, removing any cloud dependency. Each capability operates as an independent node, forming a distributed intelligence architecture where individual modules can be developed, tested, and replaced without affecting the rest of the system.

C. Neural Network-Based Kinematics

Save [7] trained separate forward and inverse kinematics neural networks on simulated Kinova Gen3[1] 7-DoF arm data. Because the inverse model alone lacked sufficient accuracy, an iterative correction loop was introduced in which the forward network validates each inverse prediction and, if the error exceeds a threshold, feeds the predicted joint angles back as a refined starting point. The forward kinematics network predicted end-effector position with a median error of 2.68 cm, and when paired with the inverse kinematics model through this loop, the system reached target poses with a median error of 9.95 cm. These results inform the planned integration of ANN-based solvers into REACH; the architecture is designed so that such solvers can replace deterministic API calls without affecting the rest of the system.

[1] Kinova Gen3 robots, `https://www.kinovarobotics.com/product/gen3-robots`, accessed August 2025.

D. Prompt Engineering for Robotic Command Generation

Marvin et al. [8] described prompt engineering as the set of strategies used to shape LLM behavior through structured input design, including techniques such as few-shot exemplars and output format indicators that guide models toward intended responses. In the context of robotic systems, prompt engineering enables the translation of free-form natural language into machine-interpretable command formats. In aircraft engineering, da Cruz Silva et al. [9] found that a RAG-augmented LLM matched or exceeded the accuracy of zero-shot and in-context prompted models on a specialized aircraft design question-answering dataset, particularly for multi-hop reasoning tasks. For satellite ground testing, Connolly and Anderson [10] applied GPT-4 to subsystem documentation and testing workflows, reporting useful performance for requirements refinement but finding that autonomously generated test scripts were largely unreliable without extensive human supervision.

However, these applications use prompt engineering to generate human-readable text such as answers and documentation. REACH applies prompt engineering to a different end: constraining a locally hosted LLM to produce semicolon-delimited structured commands that are directly parsed and executed by a robotic system, enabling reliable multi-step task execution through a predictable parsing pipeline.

E. Object Detection and Localization

REACH requires real-time object detection and spatial localization running entirely on edge hardware. You Only Look Once (YOLO) [11] unifies detection into a single-pass neural network that simultaneously predicts bounding box coordinates and class probabilities across the full image, achieving 45 frames per second in the original implementation. YOLOv11[2] builds on this with multiple model scales, including variants compact enough for edge deployment.

For spatial localization, fiducial marker systems such as ArUco markers[3] provide robust, low-computational-overhead reference frames suitable for real-time applications. Specifically, the AprilTag fiducial system [12] offers a large, error-resilient tag library well-suited for robust detection under varied lighting and partial obstruction. By combining YOLO-based detection with fiducial marker-based coordinate estimation, REACH achieves object localization using a single monocular camera, a practical solution for resource-constrained edge computing environments.

III. System Architecture

The REACH system architecture is designed around the principle of modular, distributed edge intelligence. The current approach in developing this architecture emphasizes shifting autonomy from deterministic control pipelines to distributed, learning-enabled frameworks operating entirely on the edge. This approach has three goals:

1) Demonstrate local natural language robotic tasking.
2) Achieve low-latency, high-reliability autonomy.
3) Provide a scalable pathway to advanced edge autonomy—the ability of a system to act independently on-platform without relying on cloud or remote resources—through the establishment of a modular design where components such as kinematics solvers or vision systems can be incrementally replaced with more advanced, learning-based alternatives without requiring full system redesign.

By combining local natural language tasking, distributed processing, and modularity, this approach provides a roadmap for LLM-enabled edge autonomy in space missions. The specific hardware and software selections used to validate this architecture are detailed in Section IV.

A. System Overview

To accomplish the three primary goals outlined above, four interconnected modules compose the system, each responsible for a distinct purpose, as illustrated in Fig. 1.

- **Communication Module**: A gigabit Ethernet switch that serves as the communication backbone of the system, enabling high-speed, low-latency data exchange between modules in an air-gapped environment. By isolating traffic locally, the switch ensures full independence from external networks. This is critical for spaceflight applications where off-board connectivity may be constrained or unavailable.
- **Robotic Arm Module**: A 7-DoF robotic arm connected to an onboard compute device. This module is responsible for robotic control and kinematic execution.

[2]Ultralytics YOLOv11, `https://docs.ultralytics.com/models/yolo11/`, accessed August 2025.

[3]OpenCV ArUco marker detection, `https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html`, accessed August 2025.

- **Natural Language Translation Module**: A compute device hosting the REACH User Interface Application. This module hosts the LLM task planning system and provides an intuitive interface that allows operators to issue both text and voice commands.
- **Perception Module**: A monocular red-green-blue (RGB) camera supported by fiducial markers for establishing a coordinate reference frame and a lightweight object detection model for real-time image recognition. This module provides both recognition and positional data for detected objects, which the robotic arm module accesses to execute manipulation commands.

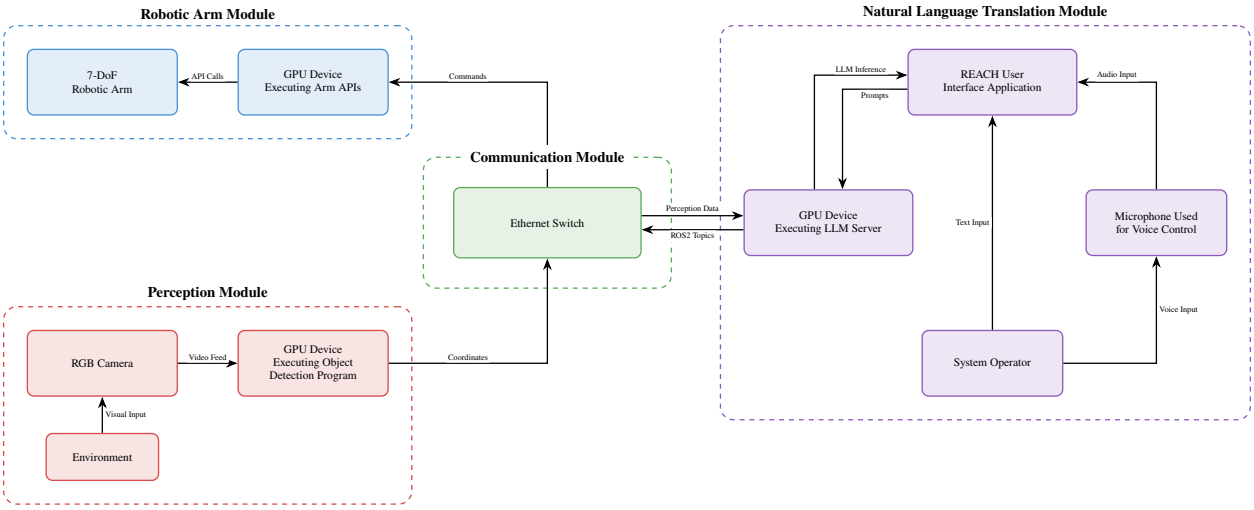


Fig. 1 REACH system architecture.

B. Communication Module

The communication module serves as the foundation of the architecture, ensuring connectivity between all other modules. Robot Operating System 2 (ROS2) [13] provides REACH its communication functionality through ROS2 topics. ROS2 topics are dedicated communication channels that enable publish-subscribe messaging between nodes. REACH publishes LLM-generated structured outputs as ROS2 topics. The robotic arm subscribes to these topics and executes commands via its Application Program Interface (API). As part of this work, custom JavaScript Object Notation (JSON)[4] commands were developed to facilitate communication with the robotic arm. JSON is a lightweight data-interchange format used here to specify the structured command interface between the translation module and the robotic arm. These custom commands are invoked through the natural language translation feature and translated into executable API commands for the arm using an internally developed programming script.

The network switch provides deterministic bandwidth allocation and reliable communication links, ensuring minimal message-passing deviations between system modules. Network isolation is further reinforced through Data Distribution Service (DDS)[5] profile configuration, which binds each ROS2 participant to a specific Ethernet interface via an interface whitelist (Appendix F). This ensures that DDS discovery and data exchange occur exclusively within the local subnet, even if individual compute modules can access additional network interfaces. This isolation from external networks is a fundamental design requirement, ensuring the system can operate entirely without off-board connectivity.

The system is designed to be modular, with the network switch enabling expansion in future iterations. Expansion may include integrating additional robotic hardware or adding new peripheral devices. Additionally, the REACH software is designed to be device-agnostic; the primary requirement for the compute modules is compatibility with ROS2. Using REACH as-is requires the specific inference tool, robotic arm API, and perception framework outlined in Section IV, but substituting any of these components requires changes only at the module level, as the ROS2 communication layer remains unchanged.

[4]JSON, `https://www.json.org`, accessed August 2025.

[5]Object Management Group Data Distribution Service, `https://www.omg.org/spec/DDS/`, accessed August 2025.

C. Robotic Arm Module

The robotic arm module provides the physical manipulation capability of the REACH system. A dedicated edge compute device runs the ROS2 subscriber node, connecting a 7-DoF robotic arm with an integrated gripper. The subscriber receives structured commands published by the Natural Language Translation module and converts them into executable arm functions through its manufacturer-provided API. Supported command types include named goal positions (Appendix C), directional motions with specified distances, gripper operations, and object manipulation commands. Object manipulation commands reference positional data that the perception module publishes for real-time target detection and localization.

Upon completing each command, the subscriber publishes an acknowledgment to a dedicated `chat/ack` topic, enabling closed-loop feedback and round-trip latency measurement. For multi-step commands, the subscriber aggregates results across all steps, logging task-level success or failure only after the entire sequence completes. This ensures reliability metrics reflect whole-task outcomes rather than individual step pass rates alone.

D. Natural Language Translation Module

The Natural Language Translation module serves as the primary interface between human operators and the robotic system. The REACH User Interface Application is an interactive chatbot integrated with ROS2 that provides this capability. The application incorporates a prompt orchestration layer for structured prompt management and session-level conversation context. Chat persistence is maintained through a local database. Three additional key features include robust data sovereignty through completely local execution, graphics processing unit (GPU)-accelerated voice control, and a user login system for local accounts on the device. The application user interface is shown in Fig. 2.

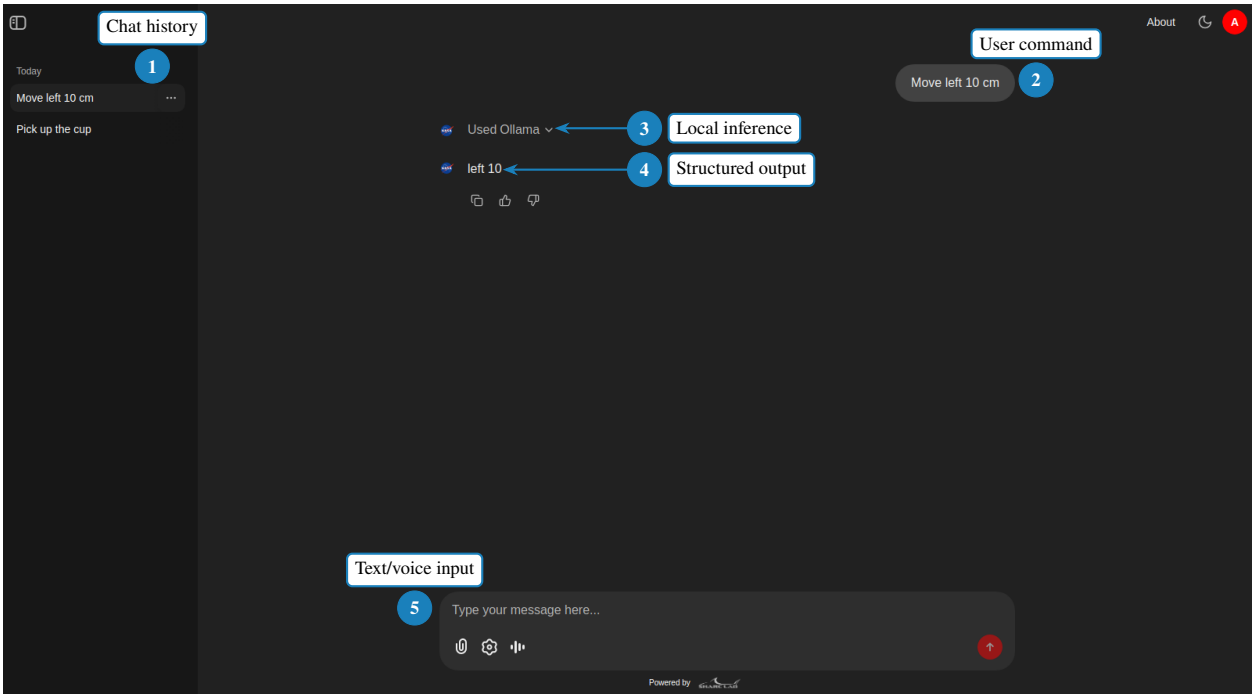


Fig. 2 REACH User Interface Application demonstrating an example translated command.

The REACH User Interface Application translates natural language commands into structured robotic procedures through prompt-engineered LLM outputs. A system prompt instructs the locally hosted LLM that it is an expert in extracting information from prompts to control a robotic arm. The system prompt constrains the model to respond with a command that specifically relates to the instruction given by the user. Multiple sequential commands are separated by semicolons to enable multi-step execution. Natural language translation examples are shown in Table 1.

These structured outputs are parsed using regular expressions and published via ROS2 topics. Each published message includes a structured metadata header of the form `[command_id:step/total:timestamp]command` (Appendix B), where `command_id` is a unique identifier, `step/total` tracks progress within multi-step sequences, and `timestamp`

Table 1 Example translations from natural language input to structured robotic commands.

User Input		Structured Output
"Go to the home position"	→	`home`
"Move forward 10 centimeters"	→	`forward 10`
"Move down 4 cm then open the claw"	→	`down 4; open`
"Pick up the bottle then go to home"	→	`pick up bottle; home`

records the send time for latency measurement. User prompts with equivalent intent translate into identical commands, ensuring consistency in the natural language processing pipeline.

E. Perception Module

The perception module performs real-time object detection and localization using a single RGB camera and fiducial markers. Camera intrinsic parameters (the intrinsic matrix K and distortion coefficients) are computed offline through a standard chessboard calibration procedure. This configuration establishes a consistent world-coordinate reference frame and eliminates the need for repeated offline calibration of the extrinsic workspace geometry. A hash map indexes each detected object with its corresponding world coordinates, allowing the robotic arm to execute commands such as "pick up the cup" by referencing the stored position of the identified object.

REACH uses a lightweight object detection model for its small footprint and compatibility with resource-constrained edge devices. Since the model is pretrained on a standard dataset, rapid prototyping was possible while leaving room for customized training in future iterations. The perception pipeline operates at 30 Hz, processing each camera frame for both fiducial marker detection and object detection. This setup supports the demonstration of perception integration in a low-budget, resource-constrained environment using a single camera, in contrast to multi-camera systems common in less constrained deployment environments.

IV. Implementation

Each REACH subsystem was implemented with specific hardware and software selections to validate the architecture described in Section III. All components were chosen to balance performance with the constraints of edge deployment, prioritizing local execution, low power consumption, and compatibility with the ROS2 communication layer.

A. Hardware Configuration

The REACH demonstration setup consists of the following hardware components:

- **Robotic Arm**: Kinova Gen3 7-DoF robotic arm with integrated gripper.
- **Arm Module Compute**: NVIDIA Jetson Orin Nano, responsible for running the ROS2 subscriber and Kinova API interface.
- **Camera**: Any generic 1080p RGB camera for monocular perception, connected to the perception module compute device.
- **Perception Module Compute**: NVIDIA Jetson Orin Nano, responsible for running the perception pipeline (object detection and fiducial marker processing) and publishing object coordinates via ROS2.
- **Natural Language Translation Module Compute**: NVIDIA Jetson AGX Orin, running the REACH User Interface Application with a locally hosted LLM for on-device inference.
- **Network**: Cisco Business 250 Smart Switch for local gigabit Ethernet communication.

All compute modules are connected to the network switch via Ethernet, forming a self-contained local network with no external connectivity.

B. Software Configuration

The software configuration comprises the following components:

- **ROS2 Humble**[6]: Middleware for inter-module communication via publish-subscribe topics, configured with

[6]ROS2 Humble Hawksbill, `https://docs.ros.org/en/humble/index.html`, accessed August 2025.

custom Fast DDS[7] profiles for interface-level network isolation.

- **Ollama**[8]: Open-source platform that enables local hosting and inference of LLMs without cloud dependencies. The default model used in REACH is Llama 3.2 1B[9] from the Llama family of models [14], a one-billion-parameter model optimized for local compute.
- **LangChain**[10]: Prompt orchestration framework providing structured prompt templates, output parsing, and conversation memory management.
- **Chainlit**[11]: LLM application interface development tool used to build the REACH User Interface Application interactive chatbot, including compatibility with multimodal audio processing and local SQLite-backed chat persistence.
- **SQLite**[12]: Lightweight, serverless database engine used for local chat persistence and user account storage, ensuring all data remains on-device.
- **Faster Whisper** (base model)[13]: CTranslate2-accelerated implementation of Whisper [15], a speech recognition model trained on 680,000 hours of multilingual audio, enabling robust local transcription without cloud dependency.
- **YOLOv11 nano** (`yolo11n.pt`): Lightweight real-time object detection model pretrained on the COCO dataset [16], selected for its small footprint and scalability across resource-constrained edge devices.
- **OpenCV**[14]: Open-source computer vision library [17] used for AprilTag 36h11 fiducial marker detection (via the ArUco API), camera calibration, and homography computation.
- **Kinova Kortex API**[15]: Manufacturer-provided API for robotic arm control and kinematic execution, accessed via protobuf-based[16] session management.

C. Prompt Engineering for Task Structuring

As described in Section III, the REACH system uses prompt engineering to constrain LLM outputs into structured, parsable commands. The system prompt specifies that the model must respond with one of the following output types: a named goal position, a directional motion with magnitude, a gripper command (open or close), or an object manipulation command. By producing structured, semicolon-delimited command strings, the system ensures consistent and repeatable translations that enable reliable parsing by downstream regular expression processors, minimizing the risk of malformed commands reaching the robotic execution layer. The complete system prompt used to focus output can be found in Appendix D. This approach enables flexible, natural language-driven tasking while maintaining the structured outputs necessary for reliable robotic execution, bridging the gap between the structure of traditional deterministic control and the adaptability required for unstructured mission environments.

D. Voice Control and Transcription

REACH supports both typed and spoken commands. Within Chainlit, multimodal features enable real-time audio processing. The microphone interface captures audio input, and the system monitors for speech activity through audio energy thresholds. When the audio energy falls below a silence threshold for a configurable timeout duration, the accumulated speech segment is processed.

Audio segments are converted to WAV format (24 kHz, mono, 16-bit PCM) and processed using the SpeechRecognition[17] module with Faster Whisper running locally. Audio segments shorter than 1.71 seconds are rejected to avoid processing noise or incomplete utterances. This approach ensures functional transcription while maintaining data sovereignty and eliminating cloud dependencies. The transcribed text is then passed to the LLM translation pipeline identically to typed input, providing a consistent processing path regardless of input modality.

To minimize first-inference latency, both the Ollama LLM and the Faster Whisper speech-to-text (STT) model are pre-loaded into GPU memory at application startup through warm-up inferences on trivial inputs. This eliminates

[7] eProsima Fast DDS, `https://github.com/eProsima/Fast-DDS`, accessed August 2025.
[8] Ollama, `https://ollama.com`, accessed August 2025.
[9] Llama 3.2 1B, `https://huggingface.co/meta-llama/Llama-3.2-1B`, accessed August 2025.
[10] LangChain, `https://github.com/langchain-ai/langchain`, accessed August 2025.
[11] Chainlit, `https://github.com/Chainlit/chainlit`, accessed August 2025.
[12] SQLite, `https://www.sqlite.org`, accessed August 2025.
[13] Faster Whisper, `https://github.com/SYSTRAN/faster-whisper`, accessed August 2025.
[14] OpenCV, `https://opencv.org`, accessed August 2025.
[15] Kinova Kortex API, `https://github.com/Kinovarobotics/kortex`, accessed August 2025.
[16] Protocol Buffers, `https://protobuf.dev`, accessed August 2025.
[17] SpeechRecognition, `https://github.com/Uberi/speech_recognition`, accessed August 2025.

cold-start delays of 15–30 seconds that would otherwise occur on the first real command, ensuring that all reported latency measurements reflect steady-state performance.

E. Object Detection and Localization

REACH employs a setup in which object detection and localization rely on a single RGB camera rather than the multi-camera or RGB-D configurations common in larger systems, keeping the hardware footprint minimal at this stage of development. This aligns with the broader goal of REACH to operate effectively in resource-constrained environments. Fig. 3 illustrates the workspace layout, including fiducial marker placement, camera positioning, and an example detected object.

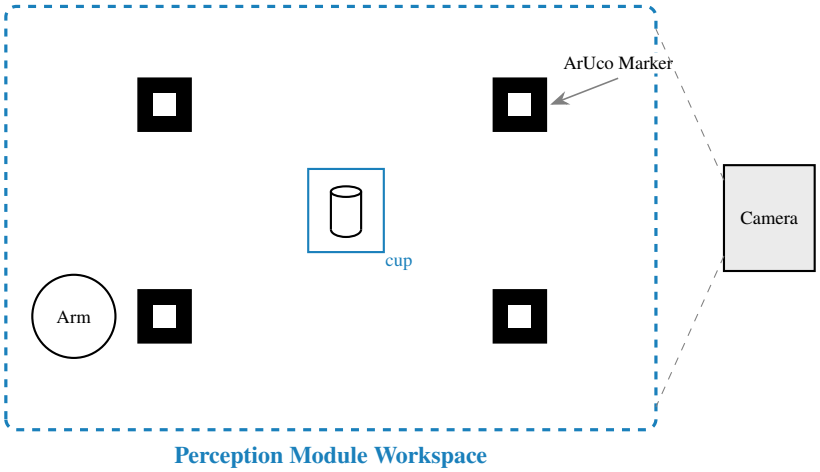


Fig. 3 Perception module workspace layout: ArUco fiducial markers establish the world-coordinate reference frame, while YOLOv11 detects objects (e.g., cup) through a monocular RGB camera.

1. Fiducial Marker Reference Frame

To establish a world-coordinate reference frame on the planar workspace surface, the system leverages fiducial markers from the AprilTag 36h11 dictionary, accessed through the OpenCV ArUco API. These markers are binary square fiducials that can be robustly detected under various lighting conditions and viewing angles. The AprilTag 36h11 family is well-suited for real-time applications due to its low computational overhead, large tag capacity, and error-resilient encoding, allowing reliable detection even under partial obstruction.

Let the base of the robotic arm define the origin $\mathbf{O} = [0, 0, 0]^T$ of the coordinate system. Consider a fixed set of fiducial markers $\mathcal{A} = \{A_m \mid m \in \mathbb{N},\ 1 \leq m \leq 4\}$. During initialization, the center of each marker is manually assigned real-world coordinates (in meters):

$$A_m = \begin{bmatrix} x_m \\ y_m \\ 0 \end{bmatrix} \tag{1}$$

relative to $\mathbf{O}$. These positions are used to compute the homography matrix:

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \tag{2}$$

which maps a point (u, v) in image coordinates to the corresponding world coordinates (x, y, z). Marker detection is implemented using the OpenCV `ArucoDetector` class and its associated `detectMarkers()` function to extract the pixel-space corners of each A_m. The marker center is computed as the centroid of the detected corners: $A_m = (u_m, v_m)$.

2. YOLO-Based Object Detection

YOLOv11 performs real-time object detection on the camera feed. For each detected object δ_c, the bottom center point of the associated bounding box is selected as the representative contact point with the surface. This point is expressed in homogeneous image coordinates:

$$\delta_{I,c} = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \tag{3}$$

where I denotes the image reference frame. The world coordinates of the detected object are approximated through the homographic transformation:

$$\delta_W = \begin{bmatrix} x \\ y \\ w \end{bmatrix} = H \cdot \delta_{I,c} \tag{4}$$

The planar world position is obtained by normalizing: $(x/w,\ y/w)$. Since δ_W is expressed with respect to the world coordinate system aligned with the robotic arm base frame, the detected objects can be directly manipulated. Each detected object is indexed in a hash map keyed by its class label, enabling commands such as "pick up the cup" to be mapped to stored world coordinates.

F. Execution and Kinematics

The Kinova arm subscribes to published ROS2 command topics and executes them via its API. The Kinova Kortex API communicates with the arm through a protobuf-based session over a dedicated Ethernet connection. Upon session initialization, the subscriber establishes a transport layer connection and authenticates with the arm controller, after which it translates each parsed command into the corresponding API call, including goal position actions, Cartesian incremental motions, or gripper open or close commands. Custom JSON commands were developed using the Kinova API framework as a reference, wrapping the low-level protobuf calls into higher-level command abstractions callable through the REACH natural language translation functionality. While this implementation uses the Kinova Gen3 robotic arm, the robotic arm module in the architecture is designed generically: substituting a different arm requires only replacing the API translation layer while the upstream ROS2 command interface remains unchanged.

G. Command Execution Pipeline

The end-to-end pipeline transforms a user command into physical robotic motion through seven stages, illustrated in Fig. 6 (Appendix A). Typed input proceeds directly to inference, while spoken input is first transcribed via Faster Whisper. The LLM hosted on the REACH User Interface Application then translates the text into a structured, semicolon-delimited command string. Regular expressions parse the output into individual command tokens, which are published as ROS2 messages. For manipulation commands referencing named objects, the subscriber queries the perception hash map for the target world coordinates before the Kinova arm executes the sequence via the API. The full command dispatch logic is provided in Appendix E.

V. Results

The REACH system was evaluated across five performance dimensions: end-to-end latency, task execution reliability, natural language translation accuracy, perception performance, and robustness to internal faults. A set of example commands supplements the quantitative analysis, mapping user inputs to executed robotic actions across both single-step and multi-step scenarios.

A. End-to-End Latency

Latency was measured for the *command processing pipeline*, defined as the interval from user input through ROS2 message publication on the Natural Language Translation module. Arm actuation time—the physical execution duration from ROS2 message receipt to movement completion—ranged from approximately 1 second for gripper operations to 3–6 seconds for position navigation and multi-step manipulation sequences. These durations were not included in results as arm actuation time is not dependent upon the REACH processing pipeline; however, the system logs actuation duration per-step as part of its closed-loop outcome tracking capability (Appendix B).

To ensure reported latencies reflect steady-state performance, an outlier filter excludes cold-start entries where the observed latency exceeds three times the median:

$$t_i \in \mathcal{D}_{\text{valid}} \iff t_i \leq 3 \cdot \tilde{t}, \quad n \geq 3 \tag{5}$$

where $\tilde{t}$ is the sample median. This filter supplements the warm-up inference described in Section IV; while warm-up eliminates the primary cold-start penalty, the outlier filter provides a secondary safeguard against lingering initialization effects. This criterion removed 15–30 second first-inference outliers while preserving latency distribution. Additionally, cross-device timestamps between the Natural Language Translation and arm module Jetsons are corrected for a measured Network Time Protocol (NTP) clock offset:

$$t^*_{\text{del}} = (t_{\text{recv}} - t_{\text{send}}) - \Delta_{\text{clock}}, \quad \Delta_{\text{clock}} = 0.011 \text{ s} \tag{6}$$

Table 2 presents the command processing pipeline latency. For text-based input, the pipeline completes in an average of 0.642 seconds (median 0.563 s), with LLM inference accounting for 98.7% of the total and command parsing plus ROS2 publication contributing the remaining 1.3% at 0.008 seconds. For voice input, the pipeline includes an additional speech-to-text stage with a latency of 1.300 seconds on average (median 1.200 s, range 0.457–5.754 s), yielding a measured end-to-end voice pipeline latency of 2.396 seconds on average (median 1.622 s). The voice total is measured end-to-end over the 35 voice trials, while the LLM inference average is computed across all 243 trials; because LLM inference latency varied between trial sets, the component averages do not sum to the voice total.

Table 2 Command processing pipeline latency breakdown.

Pipeline Stage	Avg. Latency, s	Median, s	n
Speech-to-Text (Faster Whisper)	1.300	1.200	35
LLM Inference (Llama/Ollama)	0.634	0.555	243
Command Parse + ROS2 Publish	0.008	0.008	243
Total (Text Input)	**0.642**	**0.563**	243
Total (Voice Input)	**2.396**	**1.622**	35

ROS2 delivery latency (publish to receive) averaged 0.034 seconds (median 0.032 s), corrected for the clock offset using Eq. (6). These results demonstrate that the command processing pipeline achieves sub-second latency for text input and a median of 1.622 seconds for voice input, with LLM inference dominating the text pipeline time.

B. Task Execution Reliability

Task execution reliability was evaluated for both single-step commands and multi-step tasks. Single-step commands consist of atomic actions such as directional movement, gripper operations, or navigation to named positions. Multi-step tasks consist of sequences of two or more atomic commands parsed from semicolon-delimited LLM outputs.

Table 3 Task execution reliability by command type.

Command Type	Trials	Successes	Rate, %
Single-step command	175	172	98.3
Multi-step task (2 steps)	67	60	89.6
Multi-step task (3+ steps)	47	39	83.0
Overall multi-step	114	99	86.8

Single-step commands achieved a 98.3% success rate across 175 trials, demonstrating high reliability for standalone operations. Multi-step tasks achieved an overall success rate of 86.8% across 114 trials, with 2-step tasks at 89.6% and 3+ step tasks at 83.0%. The observed decrease with increasing step count is attributable to error propagation across sequential steps. This effect is most pronounced in perception-dependent manipulation tasks where object localization errors compound through successive actions.

C. Natural Language Translation Accuracy

The accuracy of the natural language translation pipeline was evaluated by comparing LLM-generated structured outputs against expected command formats for a set of test prompts. To provide a per-category assessment, multi-step commands were decomposed into individual sub-commands, yielding aggregate accuracy across all command categories.

A key finding is that user prompts with equivalent semantic intent consistently translated into identical structured commands, confirming that the prompt engineering approach produces deterministic and repeatable translations. Fig. 4 presents aggregate translation accuracy across all command categories.

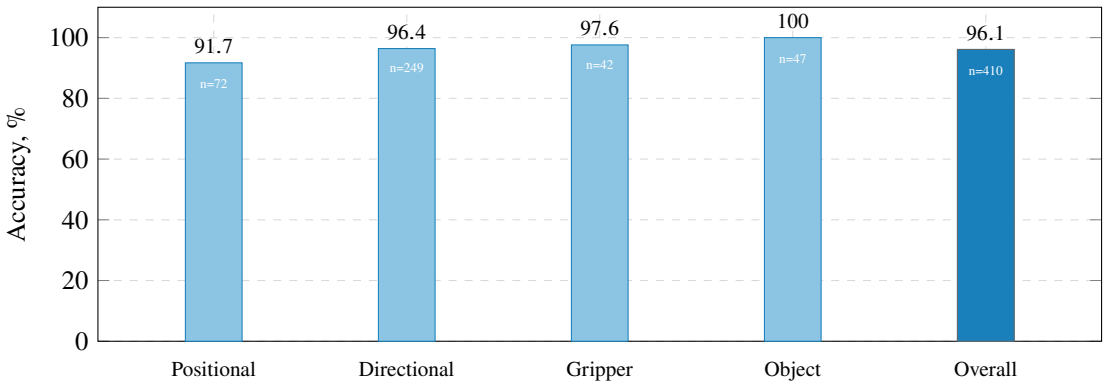


Fig. 4 Aggregate natural language translation accuracy by command category.

D. Perception Performance

Perception performance was evaluated in terms of object detection rate and localization precision. Localization accuracy was assessed by comparing estimated world coordinates against known ground-truth positions where objects were repeatedly placed at fixed locations during testing. Mean localization error, in centimeters, is computed as the Euclidean distance between the estimated and ground-truth positions:

$$\epsilon_i = \sqrt{(\hat{x}_i - x_i)^2 + (\hat{y}_i - y_i)^2} \times 100 \tag{7}$$

where $(\hat{x}_i, \hat{y}_i)$ and (x_i, y_i) are the estimated and ground-truth coordinates in meters, respectively.

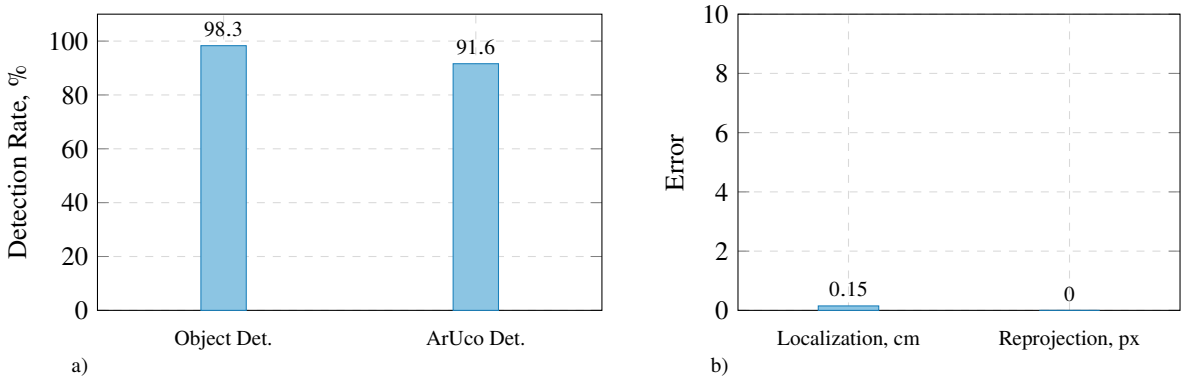


Fig. 5 Perception module performance: a) detection rates; b) localization and reprojection errors.

Perception was evaluated using task-windowed metrics, restricting analysis to a ±10-second window around pick-up commands ($n = 2,420$ frames). Within these active manipulation windows, object detection rate was 98.3% and ArUco marker detection rate was 91.6%, demonstrating high perception reliability during task execution. Mean localization error was 0.15 cm, computed using fixed object placements at known ground-truth positions. Homography re-projection error was 0.000 px; because the homography is computed from the same marker correspondences used for evaluation,

this metric confirms numerical correctness of the transform rather than localization accuracy, which is instead quantified by the mean localization error. Fig. 5 summarizes these results.

E. Robustness to Internal Faults

System robustness was evaluated under simulated internal communication faults using `iptables` with the `statistic` module for packet drops and the `hashlimit` module for bandwidth restriction. Fault conditions included injected packet drops at 1%, 5%, and 10% loss rates; bandwidth restriction to 256 kbps, 1,000 kbps, and 10,000 kbps; a combined fault (5% packet drops with 1,000 kbps bandwidth restriction); and a recovery round after fault removal. Each condition was tested with 30 single-step command trials. Robustness testing confirmed that the system achieved 100% task success across all 270 trials, with no degradation in reliability under any tested condition. Latency impact was negligible for bandwidth restriction and low packet-drop rates; 10% packet drops produced a 31.1% increase in average command latency (from 1.538 s to 2.015 s), though task success was unaffected. The system also recovered fully to nominal performance immediately after faults were removed.

Table 4 System performance under simulated internal fault conditions.

Fault Condition	Trials	Success, %	Avg Latency, s	Latency Impact
Nominal (no faults)	30	100.0	1.538	Baseline
Packet drops (1%)	30	100.0	1.536	−0.1%
Packet drops (5%)	30	100.0	1.629	+6.0%
Packet drops (10%)	30	100.0	2.015	+31.1%
Packet drops (all)	*90*	*100.0*	*1.727*	*+12.3%*
Bandwidth 256 kbps	30	100.0	1.534	−0.2%
Bandwidth 1,000 kbps	30	100.0	1.550	+0.8%
Bandwidth 10,000 kbps	30	100.0	1.540	+0.2%
Bandwidth (all)	*90*	*100.0*	*1.541*	*+0.3%*
Combined (5% drops + 1 Mbps)	30	100.0	1.551	+0.9%
Recovery (faults removed)	30	100.0	1.536	−0.1%

F. Example Commands

To contextualize quantitative results, a set of example commands was executed. These illustrate the capability of the system to handle both simple and compound inputs. Object detection tasks successfully enabled the arm to pick up items identified by name, using YOLOv11 detection and fiducial-based localization to autonomously determine grasp positions. Table 5 lists commands spanning single-step, multi-step, and perception-dependent operations.

Table 5 Example commands and executed actions.

User Command	Executed Actions
“Open claw”	Gripper open
“Move forward 10 cm”	Linear motion +10 cm along forward axis
“Pick up the bottle”	Detect bottle → navigate to position → grasp
“Pick up the bottle and return home”	Detect bottle → navigate to position → grasp → navigate to home position
“Move down 5 cm, close claw, move up 10 cm”	Descend → close gripper → ascend

VI. Discussion

The experimental results demonstrate that REACH achieves low-latency, reliable robotic task execution using fully local computation. These results carry implications for edge-deployed autonomy, while also revealing limitations that inform the future development of the architecture.

A. Implications of Fully Local Operation

Because all computation runs locally, REACH eliminates dependency on external connectivity, a requirement for mission environments where cloud-based compute is unavailable. The sub-second command processing pipeline latency achieved through fully local processing (of which 0.634 s is LLM inference and 0.008 s is command parsing and ROS2 publication) eliminates dependency on external network round-trip latency entirely, unlike cloud-based alternatives. With voice input, the end-to-end pipeline latency averages 2.396 seconds (median 1.622 s), which remains suitable for interactive operation. For interactive human-robot teaming scenarios, such as an astronaut issuing voice commands to a robotic arm aboard a spacecraft, this latency profile enables responsive interaction.

The fully local design of the REACH architecture also ensures data sovereignty, meaning all voice data, commands, and execution records remain within the local system. This guarantee is enforced at the middleware level: Fast DDS profiles bind each ROS2 participant to the local subnet via interface whitelisting, and chat history is persisted in a local SQLite database rather than a cloud-hosted service. This property is relevant for missions with data sensitivity constraints or where regulatory requirements preclude cloud-based data processing. While the current prototype relies on network-level isolation, the DDS middleware supports additional hardening through the Object Management Group DDS Security specification; a proposed security-hardened configuration is discussed in Section VII.

B. Comparison with Cloud-Based Systems

Compared to systems such as RT-2 [1] and PaLM-E [2], REACH operates at a fundamentally different point in the compute–connectivity design space. While RT-2 and PaLM-E demonstrate advanced multimodal reasoning, they require persistent external network connectivity to cloud infrastructure. REACH trades some degree of reasoning capability for complete network independence and predictable latency, a tradeoff that is well-suited for the communication-constrained deployment environments addressed by the architecture.

The modular architecture of REACH allows it to upgrade LLM performance as more capable edge-deployable models become available, without modifying the perception, communication, or execution subsystems. This modularity provides a pathway for REACH to incorporate more advanced language models as edge hardware capabilities and the availability of open-source models increase. Open-weight model families such as Llama [14] and Gemma [18] are rapidly expanding in both capability and efficiency, making on-device inference increasingly viable. As these models mature, the capability gap between edge-deployed and cloud-based systems continues to narrow without sacrificing the data sovereignty and connectivity independence that the REACH architecture requires.

C. Limitations

The primary limitation observed is sensitivity to perception errors, particularly in multi-step tasks where object localization accuracy directly impacts execution success. The monocular perception system, while sufficient for the commands evaluated, provides limited depth information and is susceptible to perspective distortion at the workspace periphery. Upgrading to a depth-sensing camera or a multi-camera configuration would mitigate both of these constraints.

Additionally, longer task sequences introduce increased opportunity for error propagation. A failure in any step of a multi-step task (e.g., mislocalization during a pick operation) cascades to subsequent steps, reducing overall task success rates. This behavior is reflected in the lower reliability observed for multi-step tasks relative to single-step commands. Implementing per-step verification or recovery logic would reduce this compounding effect.

The current prompt engineering approach, while effective for the command vocabulary demonstrated, requires manual extension to support new command types. As the operational vocabulary grows, maintaining the system prompt and associated parsing logic may become increasingly complex. Automating command expansion through retrieval-augmented generation would decouple the command vocabulary from manual prompt maintenance, enabling the system to scale to new domains without rewriting the translation layer.

D. Initial User Feedback

Following demonstrations of REACH to multiple engineering groups at NASA, initial qualitative feedback from users interacting with the natural language interface indicated functional usability, while also revealing instances of command ambiguity where the LLM interpretation diverged from user intent. Error recovery within a single prompt is limited; however, users were able to correct misinterpreted commands through successive prompts. These observations inform ongoing interface refinement but do not represent a formal usability evaluation. A comprehensive study incorporating task-based metrics and standardized questionnaires could quantify interface effectiveness for operational use.

E. Modularity and Extensibility

Despite the identified limitations, the modular design of REACH enables incremental improvements without requiring changes to the overall architecture. The perception module can be upgraded with an RGB-D camera or a multi-camera system to improve localization accuracy. The LLM selection can be updated to include more capable models as edge-deployable options mature. Each of these improvements can be integrated independently, validating the architectural decision to maintain strict modularity between subsystems. This design ensures that advancing any single capability does not require rearchitecting the system as a whole.

This device-agnostic, modular design is central to the long-term viability of REACH. Because each subsystem communicates through standardized ROS2 interfaces, the architecture is not tied to any specific robotic platform, sensor suite, or language model. Deploying REACH on a different robotic manipulator requires only a new execution subscriber that maps parsed commands to the target platform API; the application, perception, and communication layers remain unchanged. As edge hardware grows more capable and open-weight language models continue to improve, the architecture is positioned to integrate these advances without structural modification.

VII. Future Work

As REACH evolves, future work will focus on two primary areas: ANN-based kinematics integration and autonomous procedure generation. After these two items are addressed, there are additional areas of research applicable to REACH that may be of interest.

A. ANN-Based Kinematics Integration

The first item to be addressed in future work is replacing the inverse kinematics solver in the Kinova API with ANN-based alternatives trained in simulation and validated through hardware-in-the-loop testing. Preliminary testing during REACH development indicated that the Kinova API inverse kinematics solver occasionally failed to converge for certain target positions, motivating the exploration of ANN-based alternatives. Given existing work in ANNs for forward and inverse kinematics of the Kinova Gen3 7-DoF robotic arm [7], future work entails improving simulation fidelity in that research to provide more accurate training data. This would allow deployment of the trained neural network inverse kinematic solver on the arm, leading to more expedient and reliable kinematic solutions. This capability would be particularly beneficial in dynamic task scenarios in which API solvers may be less efficient in both speed and accuracy.

B. Autonomous Procedure Generation

The second item to be addressed in future work is iterating the REACH User Interface Application to autonomously generate multi-step robotic procedures in real-time, incorporating safety constraints and mission context awareness. This feature would allow for adaptability to dynamic environments without operator intervention, increasing reliability and supporting EIO applications. This would be further enhanced with the introduction of RAG functionality to the REACH User Interface Application, allowing for specialized responses and integration with specific command APIs that may differ between robotic platforms. The modular separation between the LLM orchestration layer and the downstream command parser means RAG can be incorporated by extending the LangChain pipeline without modifying the perception, communication, or execution subsystems. Upon completion of this work, future evaluation will assess system performance on increasingly complex multi-step tasks, such as object relocation and workspace configuration. This will include the analysis of success rates, error modes, and the extent to which autonomously generated procedures match operator intent.

C. Additional Research Directions

In addition to kinematics integration and procedure generation, a few areas of research could enhance REACH as the project matures:

- **Command schema validation**: Implementing formal JSON Schema validation [19] on the structured command objects before they are published as ROS2 topics. This would add a safety layer that rejects malformed or out-of-bounds commands before they reach the robotic arm, complementing the existing regular expression parsing with a declarative structural constraint.
- **DDS Security hardening**: Configuring a security-hardened Fast DDS setup that enables three of the built-in plugins defined by the Object Management Group DDS Security specification: PKI-based mutual authentication (`DDS:Auth:PKI-DH`), topic-level publish/subscribe access control (`DDS:Access:Permissions`), and AES-GCM-256 encryption of all topic data and metadata (`DDS:Crypto:AES-GCM-GMAC`). This configuration would include per-node X.509 certificates, a signed governance document that enforces encryption on the `chat`, `object/location`, and `chat/ack` topics, and per-participant permissions restricting each node to only the topics it requires. This would help prepare the architecture for deployment in higher-assurance environments, as the current implementation relies on network-layer isolation to protect inter-node communication.
- **Enhanced spatial perception**: Enhancing perception capabilities with Simultaneous Localization and Mapping (SLAM) for improved spatial understanding. The evaluation methodology used to compare visual and LiDAR SLAM algorithms on NASA flight test data [20], assessing robustness, accuracy, and real-time effectiveness, could guide algorithm selection for REACH. This will allow REACH to adapt to dynamic environments, making it more effective in mission scenarios, as it will no longer rely on the current planar workspace assumption. The project may also benefit from cameras with depth-sensing capabilities.
- **Further latency testing**: Characterizing latency under varying system loads, including scenarios with queued commands and rapid sequential execution. This would also include testing concurrent GPU workloads to establish throughput limits and identifying any potential sources of slower performance in the command processing pipeline. For example, this may include sending back-to-back commands while the Ollama LLM is still processing information from a prior request.
- **Human-robot teaming**: Extending REACH to support human-robot teaming in mission settings. This effort would entail researching the needs of crew members, formulating human-robot teaming scenarios applicable to mission operations, and evaluating REACH within those scenarios. Computational frameworks that model human and robotic agent performance in space habitats [21] could guide scenario development.
- **Multi-robot communication**: Configuring REACH to support multiple robotic systems on a shared local network will demonstrate scalability of the system. This effort will also allow for investigation into resource allocation and how to effectively coordinate collaborative tasks between robotic systems on edge-based networks.
- **Simulation integration**: Integrating REACH with simulation environments for real-time testing and validation. This could be alongside hardware executing tasks or before hardware deployment in order to visualize the effects of proposed hardware or software changes. This would accelerate the development timeline by streamlining analysis processes and mitigating hardware errors.

By addressing these areas, REACH will expand its role as an edge-based autonomy architecture to support a wider range of space operations.

VIII. Conclusion

REACH represents a practical step in the advancement of edge robotic autonomy, demonstrating that LLM-driven task planning, speech recognition, and object detection can be effectively integrated into a modular, distributed architecture suitable for space missions. Through fully local execution, REACH eliminates reliance on external compute resources, achieves low latency, and improves system resilience within a self-contained environment. The modularity and device-agnostic design of the architecture provide a scalable foundation for integrating additional capabilities and hardware modules as the system matures.

The implementation presented in this paper addresses the problem of enabling natural language-driven robotic control in environments where cloud connectivity is unavailable. Specifically, experimental validation demonstrates command processing pipeline latency of 0.642 seconds on average for text input and 2.396 seconds on average for voice input, with a single-step command success rate of 98.3% and a multi-step task success rate of 86.8%. Natural language translation accuracy reached 96.1% across all command categories, and object detection rate during active manipulation was 98.3%. The system operates as expected without network connectivity and performs robustly under

internal communication faults, achieving 100% task success across all tested fault conditions.

These results confirm the viability of deploying an edge-based, AI-enhanced robotic system in mission settings, specifically in communication-constrained environments such as those requiring Earth independence. Future work will extend the architecture with ANN-based kinematics and autonomous procedure generation for complex multi-step tasks. REACH validates that edge-deployed, LLM-driven robotic autonomy is not only feasible but practical for communication-constrained mission environments.

Appendix A. Command Execution Pipeline Diagram

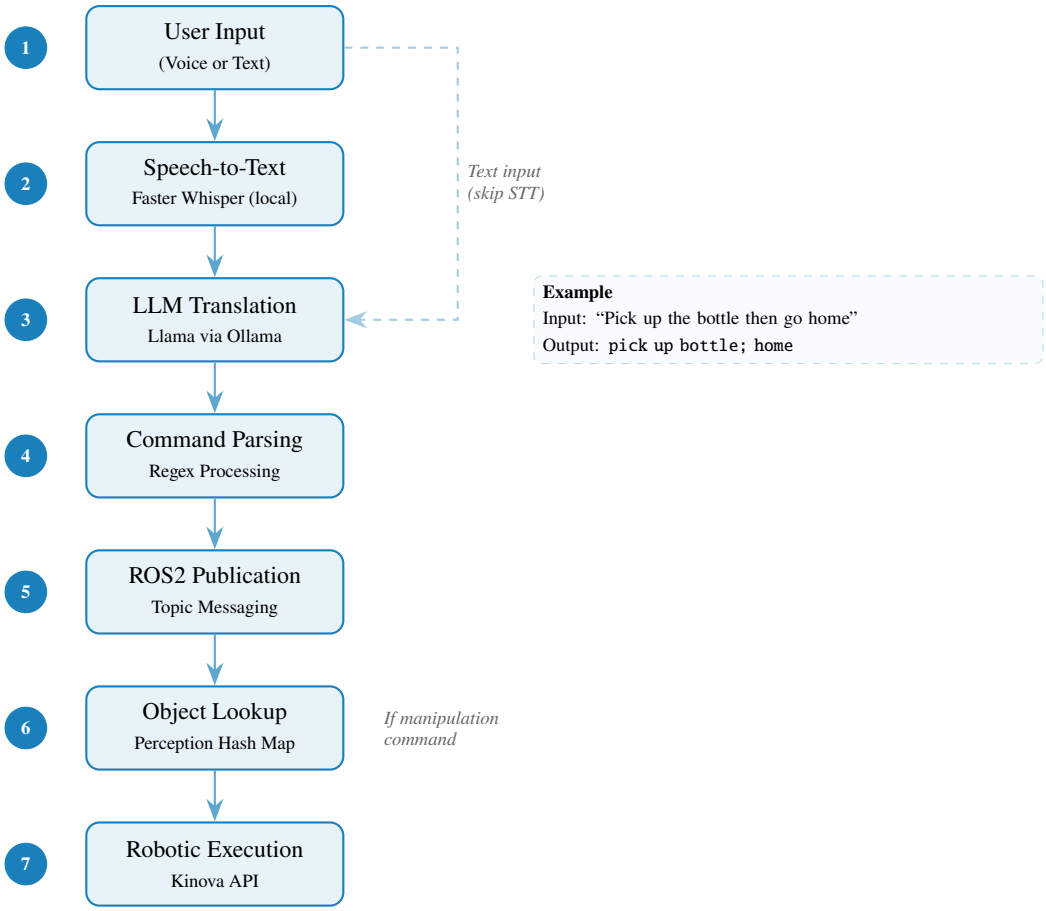


Fig. 6 Seven-stage command execution pipeline from user input to robotic actuation.

Appendix B. ROS2 Message Format

The REACH User Interface Application publishes each robot command as a `std_msgs/String` message on the ROS2 `chat` topic. Each message includes a structured metadata header:

```
[command_id:step/total:timestamp]command
```

where:

- `command_id` — a unique identifier for the command batch (e.g., `cmd_0001`).
- `step/total` — the current step index and total steps in the sequence (e.g., `1/3` for the first of three commands).
- `timestamp` — the Unix time at which the message was published, used for round-trip latency measurement.
- `command` — the structured command payload (e.g., `forward 10`, `pick up bottle`).

Example single-step command ("go to the home position"):

```
[cmd_0001:1/1:1754067600.123456]home
```

Example multi-step sequence (“pick up the bottle then go home”):

```
[cmd_0002:1/2:1754067601.482913]pick up bottle
[cmd_0002:2/2:1754067601.517246]home
```

Upon completion of each step, the robotic arm subscriber node publishes an acknowledgment on the `chat/ack` topic containing the `command_id`, success status, and actuation latency, enabling closed-loop outcome tracking.

Appendix C. Named Position Joint Angles

The Kinova Gen3 7-DoF arm uses predefined joint-angle configurations for named positions. Table 6 lists both custom and manufacturer-provided positions used by the REACH system, with angles in degrees for each of the seven actuators present on the robot (θ_1–θ_7).

Table 6 Named position joint angles (degrees) for the Kinova Gen3 7-DoF arm.

Position	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6	θ_7
Home	0.0	15.0	180.0	230.0	0.0	55.0	90.0
Rest	285.7	78.9	166.5	253.6	6.4	88.5	97.1
Up	359.6	356.2	180.4	266.6	2.0	359.7	87.7
Down	359.2	72.7	180.7	232.2	359.7	110.4	89.8
Right	33.8	15.0	180.0	230.0	360.0	55.0	90.0
Left	327.2	15.0	180.0	230.0	360.0	55.0	90.0
Base	310.7	88.7	197.6	312.4	118.0	120.3	63.1

Appendix D. LLM System Prompt

The following is the complete system prompt used in the REACH User Interface Application to constrain the active LLM to produce structured, parsable outputs from natural language input. This prompt is the primary mechanism by which free-form user commands are translated into structured command tokens, which are then post-processed and published over ROS2 to the command parsing logic described in Appendix E.

```
You are an expert in extracting information that the user states to help the user move a
robotic arm. If the user tells you to move the arm to a position, you must simply
respond with the position the user said. For example if the user tells you to move the
arm home, respond with ‘home’. Valid positions that the user could ask for are: ‘up’
‘down’ ‘right’ ‘left’ ‘home’ ‘rest’ ‘wave’. If the user asks you to move the arm in a
direction for a certain distance, respond with the direction and the distance, for
example if the user says ‘move forward 10 cm’ respond with ‘forward 10’ or if the user
asks you to move up 4 cm ‘up 4’. If the user asks you to move out 5 cm, respond with
‘out 5’. If the user asks you to move in 6 cm, respond with ‘in 6’. Motions will have a
number or a distance associated with it, so your answer should have that as well.
Respond ONLY with the user’s desired motion. Valid motions that the user could ask for
are ‘up 5’ ‘left 6’ ‘right 7’ ‘down 8’ ‘forward 5’ ‘back 9’ ‘turn right 70’ ‘rotate left
80’ ‘move out 5’ ‘move in 9’. If the user asks you to close the claw, respond with
‘close’. If the user asks you to open the claw, respond with ‘open’. The only thing you
should state in your response is the user’s desired motion. If the user asks you to
pick something up, respond ONLY with ‘pick up’ and the name of the object they state as
the command, so for example ‘pick up bottle’ ‘pick up box’ ‘pick up red pen’. If the
user tells you to do multiple motions for arm control in one prompt, such as going to a
position, turning an angle, moving a distance, split them by a semicolon, but if there’s
only one requested motion, ONLY respond with that motion, don’t respond with anything
the user did not state in their message. If the user has numbers in their prompt the
```

```
corresponding command in your response SHOULD ALSO HAVE NUMBERS. THE NUMBERS IN YOUR
RESPONSE SHOULD CORRECTLY MATCH THE NUMBERS IN THE USERS PROMPT. ONLY RESPOND WITH YOUR
COMMANDS.
```

Appendix E. Command Parsing and Dispatch Logic

The robotic arm subscriber node is implemented as a ROS2 `Node` subclass whose `self.arm` attribute wraps the Kinova API, and whose `self.motion_functions` and `self.position_functions` dictionaries map command keywords to the corresponding arm methods. The `chat_listener_callback` method receives structured LLM outputs from the ROS2 `chat` topic, parses metadata headers (command ID, step index, total steps), and dispatches commands to the appropriate arm control functions. The parsing logic handles turns/rotations, in/out motions, gripper commands, directional motions with distances, object manipulation via perception lookup, and named position navigation. Distance values received in centimeters are converted to meters before being passed to the Kinova API. After dispatch, the callback publishes an acknowledgment and tracks multi-step batch results.

```
def chat_listener_callback(self, msg: String):
    command_id, step, total, ts, payload = \
        self._parse_metadata(msg.data.strip())
    command = payload.strip().lower()
    if "turn" in command:
        angle = float(
            re.findall(r'\d+', command)[0])
        if "left" in command:
            self.arm.turn_angle(angle)
        else:
            self.arm.turn_angle(-angle)
    elif "out" in command:
        dist = float(
            re.findall(r'\d+', command)[0])
        self.arm.move_out(dist/100)
    elif "in" in command:
        dist = float(
            re.findall(r'\d+', command)[0])
        self.arm.move_out(-dist/100)
    elif "close" in command:
        self.arm.close_gripper()
    elif "open" in command:
        self.arm.open_gripper()
    elif any(c.isdigit() for c in command):
        for cmd in self.motion_functions:
            if cmd in command:
                dist = float(re.findall(
                    r'\d+', command)[0])
                self.motion_functions[cmd](
                    dist/100)
    elif "pick up" in command:
        obj = re.sub(
            r"pick up ", "", command)
        point = self.objects.get(obj, None)
        if point:
            self.arm.pick_up_object(point)
    elif command in self.position_functions:
        self.position_functions[command]()
```

Listing 1 Subscriber command dispatch logic. Acknowledgment publishing, metrics collection, and execution status propagation are omitted for clarity.

Appendix F. Fast DDS Network Profiles

REACH uses custom Fast DDS profiles to enforce network isolation. Each ROS2 participant is bound to a specific Ethernet interface via an `interfaceWhiteList`, and built-in transports are disabled so that all DDS discovery and data exchange occur exclusively on the designated local subnet. The publisher profile is used by the REACH User Interface Application node; the subscriber profile is used by the robotic arm subscriber node. The architecture supports multiple subscriber profiles (one per downstream device), while the application maintains a single publisher profile.

```
<?xml version="1.0" encoding="UTF-8" ?>
<dds>
  <profiles xmlns="http://www.eprosima.com/
      XMLSchemas/fastRTPS_Profiles">
    <transport_descriptors>
      <transport_descriptor>
        <transport_id>udp_transport</transport_id>
        <type>UDPv4</type>
        <send_buffer_size>65536</send_buffer_size>
        <receive_buffer_size>
            65536</receive_buffer_size>
        <interfaceWhiteList>
          <address><IP_ADDRESS></address>
        </interfaceWhiteList>
      </transport_descriptor>
    </transport_descriptors>
    <participant
        profile_name="publisher_participant"
        is_default_profile="true">
      <rtps>
        <useBuiltinTransports>
            false</useBuiltinTransports>
        <userTransports>
          <transport_id>
              udp_transport</transport_id>
        </userTransports>
        <defaultUnicastLocatorList>
          <locator>
            <udpv4>
              <address><IP_ADDRESS></address>
              <port><PORT_NUMBER></port>
            </udpv4>
          </locator>
        </defaultUnicastLocatorList>
      </rtps>
    </participant>
  </profiles>
</dds>
```

Listing 2 Fast DDS publisher profile. The subscriber profile is identical in structure; each node binds to its own interface address on the local subnet.

Acknowledgments

This work was funded as a Technology Investment Development (TID) project, led by the NASA Marshall Space Flight Center System Health, Automation, and Redundancy Control (SHARC) Lab, with two Amentum interns who were dedicated to this project in Summer 2025. The SHARC Lab is a research and development hub dedicated to ensuring mission success and safety by advancing the field of fault management through autonomous systems, intelligent system health management (ISHM), AI, and sensor technologies. The authors would like to acknowledge NASA MSFC EV43 members Tristan Lane (Branch Chief), Mark Wallace (Amentum Team Lead), Ben Stolz, Peter Berg, and Dr. Cameron Grace (Amentum) for their support of this project.

References

[1] Zitkovich, B., Yu, T., Xu, S., Xu, P., Xiao, T., Xia, F., Wu, J., Wohlhart, P., Welker, S., Wahid, A., Vuong, Q., Vanhoucke, V., Tran, H., Soricut, R., Singh, A., Singh, J., Sermanet, P., Sanketi, P. R., Salazar, G., Ryoo, M. S., Reymann, K., Rao, K., Pertsch, K., Mordatch, I., Michalewski, H., Lu, Y., Levine, S., Lee, L., Lee, T.-W. E., Leal, I., Kuang, Y., Kalashnikov, D., Julian, R., Joshi, N. J., Irpan, A., Ichter, B., Hsu, J., Herzog, A., Hausman, K., Gopalakrishnan, K., Fu, C., Florence, P., Finn, C., Dubey, K. A., Driess, D., Ding, T., Choromanski, K. M., Chen, X., Chebotar, Y., Carbajal, J., Brown, N., Brohan, A., Arenas, M. G., and Han, K., "RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control," *Proceedings of The 7th Conference on Robot Learning*, Proceedings of Machine Learning Research, Vol. 229, JMLR.org, Atlanta, GA, 2023, pp. 2165–2183. URL `https://proceedings.mlr.press/v229/zitkovich23a.html`.

[2] Driess, D., Xia, F., Sajjadi, M. S. M., Lynch, C., Chowdhery, A., Ichter, B., Wahid, A., Tompson, J., Vuong, Q., Yu, T., Huang, W., Chebotar, Y., Sermanet, P., Duckworth, D., Levine, S., Vanhoucke, V., Hausman, K., Toussaint, M., Greff, K., Zeng, A., Mordatch, I., and Florence, P., "PaLM-E: An Embodied Multimodal Language Model," *Proceedings of the 40th International Conference on Machine Learning (ICML'23)*, Proceedings of Machine Learning Research, Vol. 202, JMLR.org, Honolulu, HI, 2023, pp. 8469–8488. URL `https://proceedings.mlr.press/v202/driess23a.html`.

[3] Mon-Williams, R., Li, G., Long, R., Du, W., and Lucas, C. G., "Embodied Large Language Models Enable Robots to Complete Complex Tasks in Unpredictable Environments," *Nature Machine Intelligence*, Vol. 7, 2025, pp. 592–601. https://doi.org/10.1038/s42256-025-01005-x.

[4] Janbi, N., Katib, I., and Mehmood, R., "Distributed Artificial Intelligence: Taxonomy, Review, Framework, and Reference Architecture," *Intelligent Systems with Applications*, Vol. 18, 2023, p. 200231. https://doi.org/10.1016/j.iswa.2023.200231.

[5] Gala, G., Unte, T., Maia, L., Kühbacher, J., Kadusale, I., Alkoudsi, M. I., Fohler, G., and Altmeyer, S., "Safety-Critical Edge Robotics Architecture with Bounded End-to-End Latency," *arXiv preprint*, 2024. https://doi.org/10.48550/arXiv.2406.14391.

[6] Sikorski, P., Yu, K., Billadeau, L., Esposito, F., AliAkbarpour, H., and Babaiasl, M., "Improving Robotic Arms Through Natural Language Processing, Computer Vision, and Edge Computing," *Proceedings of the 3rd International Conference on Mechatronics, Control and Robotics*, Institute of Electrical and Electronics Engineers, Singapore, 2025, pp. 35–41. https://doi.org/10.1109/ICMCR64890.2025.10962987.

[7] Save, A., "Artificial Neural Networks for Solving Forward and Inverse Kinematics on a 7 Degree-of-Freedom Robotic Manipulator," *Proceedings of the 12th International Conference on Mechatronics and Robotics Engineering*, Oldenburg, Germany, 2026 (not yet published).

[8] Marvin, G., Hellen, N., Jjingo, D., and Nakatumba-Nabende, J., "Prompt Engineering in Large Language Models," *Data Intelligence and Cognitive Informatics*, Springer Nature Singapore, 2024, pp. 387–402. https://doi.org/10.1007/978-981-99-7962-2_30.

[9] Da Cruz Silva, E. A., Marsh, R., Yong, H., Middleton, S., and Sobester, A., "Retrieval-Augmented Generation and In-Context Prompted Large Language Models in Aircraft Engineering," *Proceedings of the AIAA SCITECH 2025 Forum*, AIAA, Orlando, FL, 2025, p. 0700. https://doi.org/10.2514/6.2025-0700.

[10] Connolly, B. J., and Anderson, K. M., "Harnessing Large Language Models for Satellite Ground Tests," *Proceedings of the AIAA SCITECH 2024 Forum*, AIAA, Orlando, FL, 2024, p. 0916. https://doi.org/10.2514/6.2024-0916.

[11] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A., "You Only Look Once: Unified, Real-Time Object Detection," *Proceedings of the Institute of Electrical and Electronics Engineers Conference on Computer Vision and Pattern Recognition*, Institute of Electrical and Electronics Engineers, Las Vegas, NV, 2016, pp. 779–788. https://doi.org/10.1109/CVPR.2016.91.

[12] Olson, E., "AprilTag: A robust and flexible visual fiducial system," *2011 IEEE International Conference on Robotics and Automation*, 2011, pp. 3400–3407. https://doi.org/10.1109/ICRA.2011.5979561.

[13] Macenski, S., Foote, T., Gerkey, B., Lalancette, C., and Woodall, W., "Robot Operating System 2: Design, Architecture, and Uses in the Wild," *Science Robotics*, Vol. 7, No. 66, 2022, p. eabm6074. https://doi.org/10.1126/scirobotics.abm6074.

[14] Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., et al., "The Llama 3 Herd of Models," *arXiv preprint arXiv:2407.21783*, 2024. https://doi.org/10.48550/arXiv.2407.21783.

[15] Radford, A., Kim, J. W., Xu, T., Brockman, G., Mcleavey, C., and Sutskever, I., "Robust Speech Recognition via Large-Scale Weak Supervision," *Proceedings of the 40th International Conference on Machine Learning*, Proceedings of Machine Learning Research, Vol. 202, PMLR, 2023, pp. 28492–28518. URL https://proceedings.mlr.press/v202/radford23a.html.

[16] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., and Zitnick, C. L., "Microsoft COCO: Common Objects in Context," *Computer Vision – ECCV 2014*, edited by D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars, Springer International Publishing, Cham, 2014, pp. 740–755. https://doi.org/10.1007/978-3-319-10602-1_48.

[17] Bradski, G., "The OpenCV Library," *Dr. Dobb's Journal: Software Tools for the Professional Programmer*, Vol. 25, No. 11, 2000, pp. 120–123.

[18] Team, G., Mesnard, T., Hardin, C., Dadashi, R., Bhupatiraju, S., Pathak, S., Sifre, L., Rivière, M., Kale, M. S., Love, J., Tafti, P., Hussenot, L., Sessa, P. G., Chowdhery, A., Roberts, A., Barua, A., Botev, A., Castro-Ros, A., Slone, A., Héliou, A., Tacchetti, A., Bulanova, A., Paterson, A., Tsai, B., Shahriari, B., Lan, C. L., Choquette-Choo, C. A., Crepy, C., Cer, D., Ippolito, D., Reid, D., Buchatskaya, E., Ni, E., Noland, E., Yan, G., Tucker, G., Muraru, G.-C., Rozhdestvenskiy, G., Michalewski, H., Tenney, I., Grishchenko, I., Austin, J., Keeling, J., Labanowski, J., Lespiau, J.-B., Stanway, J., Brennan, J., Chen, J., Ferret, J., Chiu, J., Mao-Jones, J., Lee, K., Yu, K., Millican, K., Sjoesund, L. L., Lee, L., Dixon, L., Reid, M., Mikuła, M., Wirth, M., Sharman, M., Chinaev, N., Thain, N., Bachem, O., Chang, O., Wahltinez, O., Bailey, P., Michel, P., Yotov, P., Chaabouni, R., Comanescu, R., Jana, R., Anil, R., McIlroy, R., Liu, R., Mullins, R., Smith, S. L., Borgeaud, S., Girgin, S., Douglas, S., Pandya, S., Shakeri, S., De, S., Klimenko, T., Hennigan, T., Feinberg, V., Stokowiec, W., hui Chen, Y., Ahmed, Z., Gong, Z., Warkentin, T., Peran, L., Giang, M., Farabet, C., Vinyals, O., Dean, J., Kavukcuoglu, K., Hassabis, D., Ghahramani, Z., Eck, D., Barral, J., Pereira, F., Collins, E., Joulin, A., Fiedel, N., Senter, E., Andreev, A., and Kenealy, K., "Gemma: Open Models Based on Gemini Research and Technology," , 2024. URL https://arxiv.org/abs/2403.08295.

[19] Pezoa, F., Reutter, J. L., Suarez, F., Ugarte, M., and Vrgoč, D., "Foundations of JSON Schema," *Proceedings of the 25th International Conference on World Wide Web*, International World Wide Web Conferences Steering Committee, Montréal, Québec, Canada, 2016, pp. 263–273. https://doi.org/10.1145/2872427.2883029.

[20] Kannan, K., Chakrabarty, A., Baculi, J. E., Kawamura, E., Holforty, W., and Ippolito, C. A., "Comparison of Visual and LiDAR SLAM Algorithms Using NASA Flight Test Data," *AIAA SCITECH 2023 Forum*, AIAA, National Harbor, MD, 2023. https://doi.org/10.2514/6.2023-2679.

[21] Liu, X., Behjat, A., Dyke, S. J., Whitaker, D., Ramirez, J., and Bilionis, I., "Computational Framework for Assessing Mission Outcomes with Humans and Robots," *AIAA Journal*, Vol. 63, No. 4, 2025, pp. 1596–1614. https://doi.org/10.2514/1.J063846.